Report #1

Artificial Neural Networks Course - Professor Kheradpisheh

Navid Mafi

Fall 2025

1 Introduction

In this assignment we will implement MLPs for biomedical image classification on the MedMNIST dataset. The objectives include understanding MLP design, activation functions, loss functions, optimizers, network depth and width, and regularization techniques. We first start by some explanations of the nature of the project/dataset and then proceed to show details and our findings.

1.1 Data

Our chosen MedMNIST subset is `BloodMNIST` (28×28 RGB images). It contains 17092 images across 8 classes. The dataset is pre-split (to train/test/-validation) sets but the split ratios don't match this assignment's request. Thus, we will aggregate and re-split them into training (70%), validation (15%), and test (15%) sets. Images are re-scaled to (0,1) and then normalized to a mean of 0 and variance of 1. Our baseline experiment applies no additional augmentation. Example images are shown in Figure 1. We will later also show and compare our model's capabilities on other subsets on the MedMNIST dataset.

1.2 Determinism

Throughout this project, a fixed random seed (42) is used to ensure reproducibility across runs. Using a constant seed guarantees that dataset

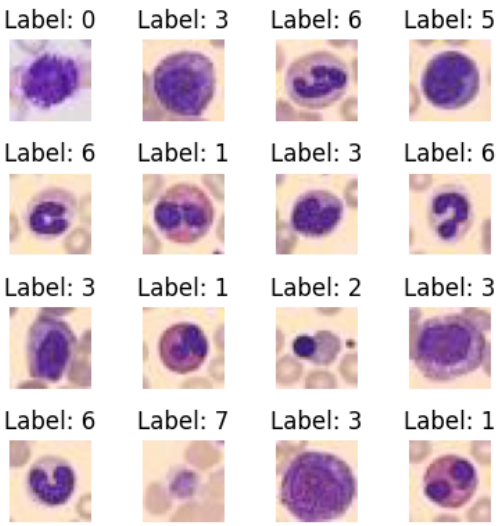

Figure 1: Example images from BloodMNIST subset.

splits, weight initializations, and stochastic operations produce identical results every time the notebook is executed. We verified that running the same notebook 10 times produces virtually identical results, with the standard deviation of the loss across runs being less than 0.01%. We believe that suffices for the verifiability of our work.

GPU operations, particularly those using CUDA libraries such as cuBLAS and cuDNN, are not fully deterministic by default and can produce slightly different results due to parallelism and atomic operations. To enforce deterministic behavior, CUDA calls were made blocking, at the cost of some performance. Furthermore, all PyTorch PRNG states are reset before each initialization, including seeds for weight initializers such as Kaiming normal, resulting in reproducible loss and accuracy values for repeated experiments. We use the seed number 42 throughout the project.

1.3 Methodology

We label each experiment with a self-descriptive ID. All experiments are trained for 100 epochs on the dataset associated with that experiment group. For every run, we report:

- Training and validation loss and accuracy curves.

- A summary table containing key metrics:
  - Final training and validation accuracy and loss.
  - Number of trainable parameters in the model.
  - Optimizer used.
  - Loss function employed.
  - Learning rate scheduler (if any) and its configuration.

2 MLP Implementation

The MLP architecture is Flatten → 128 → 64 → C with ReLU activations, Kaiming initialization, and no dropout or batch normalization in the baseline, as requested in the task. The parameters we used for our training such as batch size, learning rate, etc are summarized in our experiment is shown in table 1.

We report the results as follows:

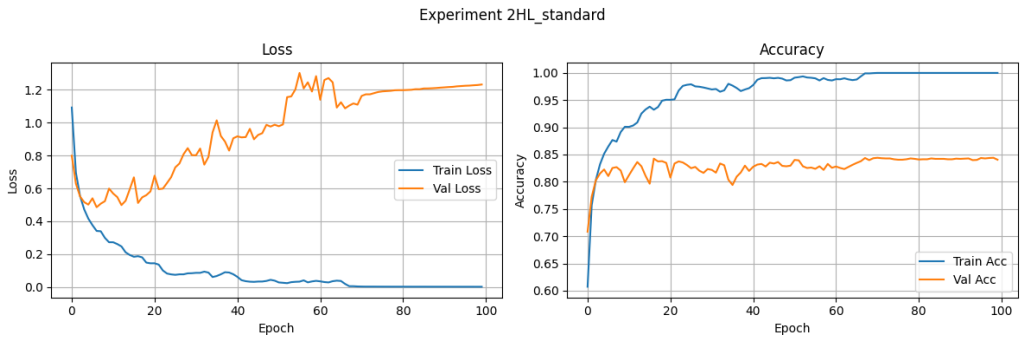

Figure 2: 2HL MLP + ReLU Accuracy/Loss plot

Metric	Value
Activation	ReLU
Batch Size	256
Loss	CrossEntropyLoss()
Optimizer	Adam
Optimizer Params	LR=0.001
Param Count	310090
Regularization	None
Scheduler	None
Train Acc	99.98%
Train Loss	0.0
Train Time (sec.)	19.84
Training Samples	11964
Val Acc	84.05%
Val Loss	1.23

Table 1: 2HL MLP + ReLU Experiment Parameters

Observations: Right off the bat, we see that the network is overfitting (note the large gap between training training/val loss and accuracy which only seems to grow bigger). This network only has 2 hidden layers and about 310k params but the dataset is also computationally simple. Our hypothesis is that this shows that our model is disproportionately more complex than our data. We later see that using regularization techniques like dropout improve these results very significantly.

3 Experiments

In order to sanity-check ourselves we did try reducing batch size quite a bit to see whether we'd see any improvements without techniques like dropout.

Even after lowering the batch size to 64, the (un)generalization situation remains the same. Training accuracy still saturates near 100% and validation accuracy remains stuck around a similar ceiling. See the training plots for a more intuitive look.

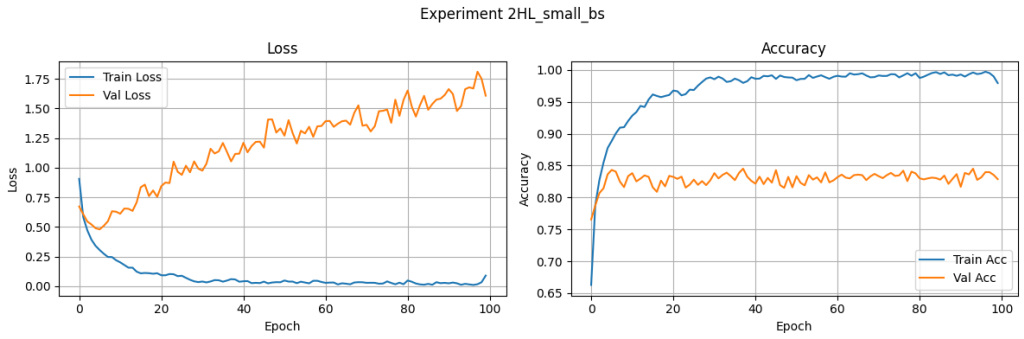

Figure 3: 2HL MLP + ReLU BS=64 Accuracy/Loss plot

Metric	Value
Activation	ReLU
Batch Size	256
Loss	CrossEntropyLoss()
Optimizer	Adam
Optimizer Params	LR=0.001
Param Count	310090
Regularization	None
Scheduler	None
Train Acc	97.94%
Train Loss	0.09
Train Time (sec.)	55.94
Training Samples	11964
Val Acc	82.88%
Val Loss	1.61

Table 2: 2HL MLP + ReLU BS=64 Experiment Parameters

3.1 Activation Functions

3.1.1 ReLU

We have seen ReLU's performance above. Using that as a reference, we will now compare that with other activation functions.

3.1.2 LeakyReLU

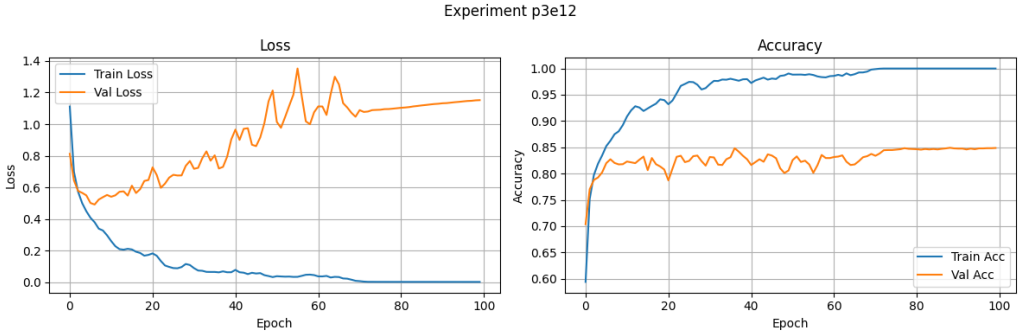

Figure 4: LeakyReLU Activation Accuracy/Loss plot

Metric	Value
Activation	LeakyReLU
Batch Size	256
Loss	CrossEntropyLoss()
Optimizer	Adam
Optimizer Params	LR=0.001
Param Count	310090
Regularization	None
Scheduler	None
Train Acc	99.98%
Train Loss	0.0
Train Time (sec.)	20.57
Training Samples	11964
Val Acc	84.87%
Val Loss	1.15

Table 3: LeakyReLU Activation Experiment Parameters

Observations: LeakyReLU behaves almost identically to ReLU in this setup and we think that's because of the depth of the MLP (or lackthereof). In terms of convergence, we are still overfitting and not generalizing well. LeakyReLU doesn't do much to improve this situation specifically. Validation accuracy increasese by only 1% which is well within margin of error. Training is a bit slower.

3.1.3 Tanh

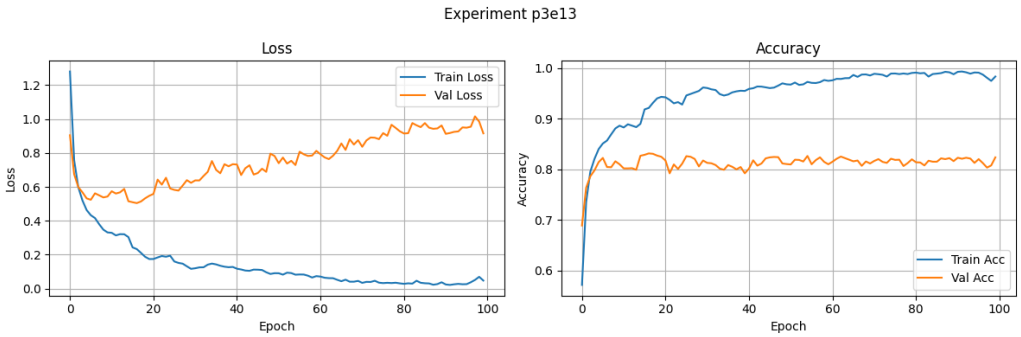

Figure 5: Tanh Activations Accuracy/Loss plot

Metric	Value
Activation	Tanh
Batch Size	256
Loss	CrossEntropyLoss()
Optimizer	Adam
Optimizer Params	LR=0.001
Param Count	310090
Regularization	None
Scheduler	None
Train Acc	98.37%
Train Loss	0.05
Train Time (sec.)	22.06
Training Samples	11964
Val Acc	82.37%
Val Loss	0.92

Table 4: Tanh Activations Experiment Parameters

Observations: Tanh does not demonstrate a big benefit over our previous methods. However it performs slightly better, and we could attrivute that to the fact that we have zero-centered outputs. It still lags behind ReLU and LeakyReLU (3% difference in validation accuracy).

3.1.4 Sigmoid

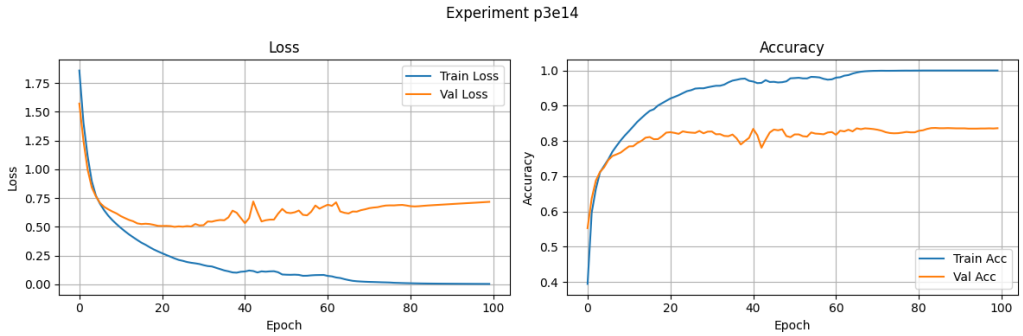

Figure 6: Sigmoid Activation Accuracy/Loss plot

Metric	Value
Activation	Sigmoid
Batch Size	256
Loss	CrossEntropyLoss()
Optimizer	Adam
Optimizer Params	LR=0.001
Param Count	310090
Regularization	None
Scheduler	None
Train Acc	99.97%
Train Loss	0.0
Train Time (sec.)	23.96
Training Samples	11964
Val Acc	83.62%
Val Loss	0.72

Table 5: Sigmoid Activation Experiment Parameters

Observations: Sigmoid underperforms compared to all of the previous methods. Its training is slow, and the network shows some vanishing gradient behavior.

3.2 Optimizers Experiments

In part 2, we used the Adam optimizer with lr=0.001. In this part we show the performance of three other optimizers on the same part 2 MLP. Those three optimizers are SGD (no momentum), SGD (momentum=0.9), and RMSProp.

3.2.1 SGD (no momentum)

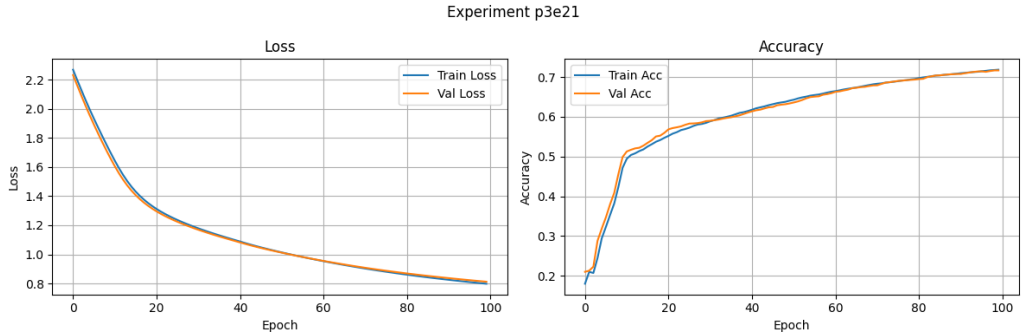

Figure 7: SGD Accuracy/Loss plot

Metric	Value
Activation	ReLU
Batch Size	256
Loss	CrossEntropyLoss()
Optimizer	SGD
Optimizer Params	LR=0.001
Param Count	310090
Regularization	None
Scheduler	None
Train Acc	71.86%
Train Loss	0.8
Train Time (sec.)	21.25
Training Samples	11964
Val Acc	71.68%
Val Loss	0.81

Table 6: SGD Experiment Parameters

Observations: The final training and validation accuracies are similar, which indicates that the model is not overfitting but it's also not extracting meaningful structure from the data. Overall, the experiment shows that in this case, SGD without momentum is stable, but inefficient, and reaches a suboptimal plateau well below other methods.

3.2.2 SGD (0.9 momentum)

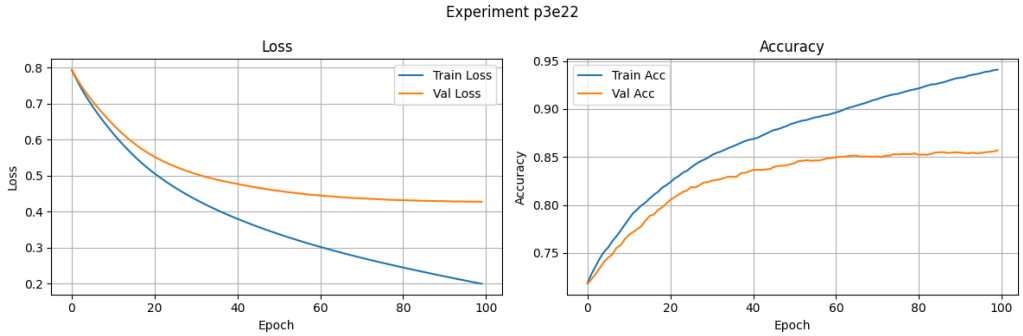

Figure 8: SGD momentum Accuracy/Loss plot

Metric	Value
Activation	ReLU
Batch Size	256
Loss	CrossEntropyLoss()
Optimizer	SGD
Optimizer Params	LR=0.001momentum=0.9
Param Count	310090
Regularization	None
Scheduler	None
Train Acc	94.08%
Train Loss	0.2
Train Time (sec.)	27.07
Training Samples	11964
Val Acc	85.69%
Val Loss	0.43

Table 7: SGD momentum Experiment Parameters

Observations: Adding momentum (0.9) to SGD improves convergence noticeably. Training accuracy rises to 82% and validation accuracy reaches 79%, a significant increase over vanilla SGD. Loss curves are smoother and descend more steadily.

3.2.3 RMSprop

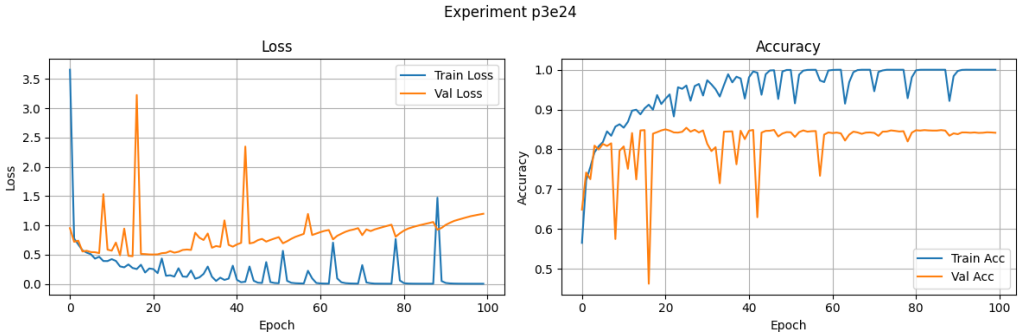

Figure 9: RMSprop Accuracy/Loss plot

Metric	Value
Activation	ReLU
Batch Size	256
Loss	CrossEntropyLoss()
Optimizer	RMSprop
Optimizer Params	LR=0.001
Param Count	310090
Regularization	None
Scheduler	None
Train Acc	99.98%
Train Loss	0.0
Train Time (sec.)	23.67
Training Samples	11964
Val Acc	84.17%
Val Loss	1.2

Table 8: RMSprop Experiment Parameters

Observations: RMSProp shows fast training convergence, with the final training accuracy reaching 99% and a loss of 0.03. Validation performance though, lags at 83% accuracy with a loss of 0.73, which indicates significant overfitting. The loss and accuracy curves are extremely jagged.

3.2.4 Learning rate effects

Learning rate is the scalar multiplier that determines the size of the step we take in the direction of the negative gradient during each parameter update. If set too high (e.g 1.0) the model will diverge and oscillate, and won't reach a good minima. If set too low (too close to zero) the model will converge extremely slowly. It might get stuck in a a local minimum because it doesn't have the "energy" to escape. Of course accuracy is only possible in models

that converge to good minimas. Sensible values for LR are around 10^{-3} depending on other parameters like BS, the optimizing algorithm, scheduling, etc. We conduct all of our experiments with LR=0.001 unless otherwise specified.

3.3 Depth Experiments

In each experiment, the hidden depth are denoted by the numbers in parentheses.

3.3.1 Shallow

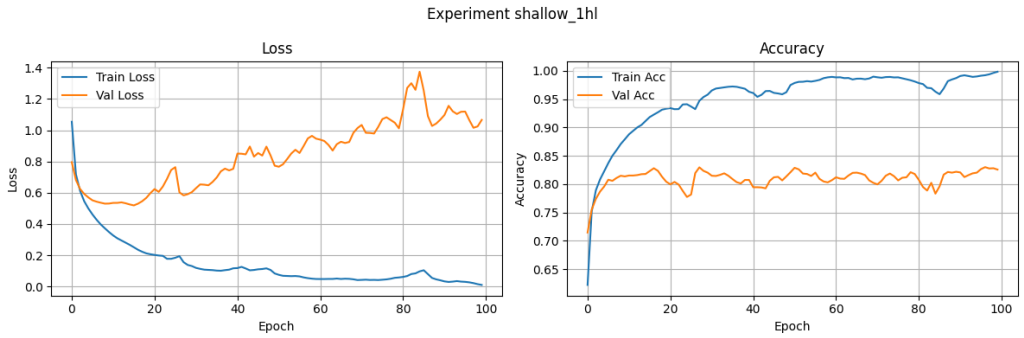

Figure 10: Shallow 1HL (64) Accuracy/Loss plot

Metric	Value
Activation	ReLU
Batch Size	256
Loss	CrossEntropyLoss()
Optimizer	Adam
Optimizer Params	LR=0.001
Param Count	151242
Regularization	None
Scheduler	None
Train Acc	99.84%
Train Loss	0.01
Train Time (sec.)	19.34
Training Samples	11964
Val Acc	82.57%
Val Loss	1.07

Table 9: Shallow 1HL (64) Experiment Parameters

Observations: Despite the significantly reduced parameter count (151k, reduced by half), the model still overfits aggressively!

3.3.2 Medium

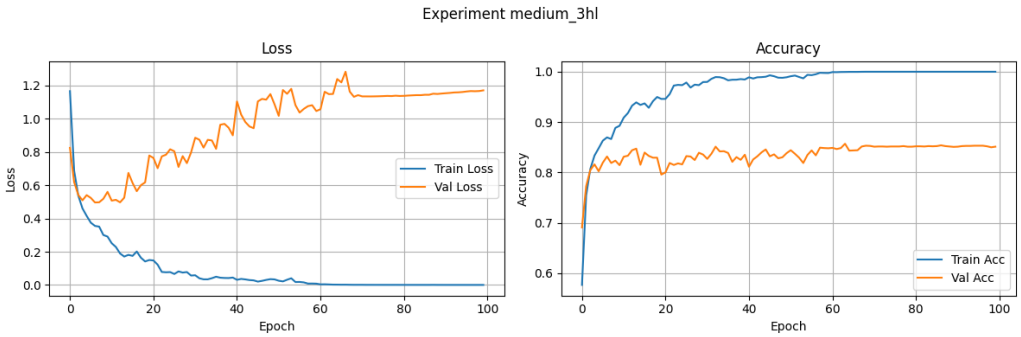

Figure 11: Medium 3HL (256,128,64) Accuracy/Loss plot

Metric	Value
Activation	ReLU
Batch Size	256
Loss	CrossEntropyLoss()
Optimizer	Adam
Optimizer Params	LR=0.001
Param Count	644170
Regularization	None
Scheduler	None
Train Acc	99.98%
Train Loss	0.0
Train Time (sec.)	23.71
Training Samples	11964
Val Acc	85.10%
Val Loss	1.17

Table 10: Medium 3HL (256,128,64) Experiment Parameters

3.3.3 Deep

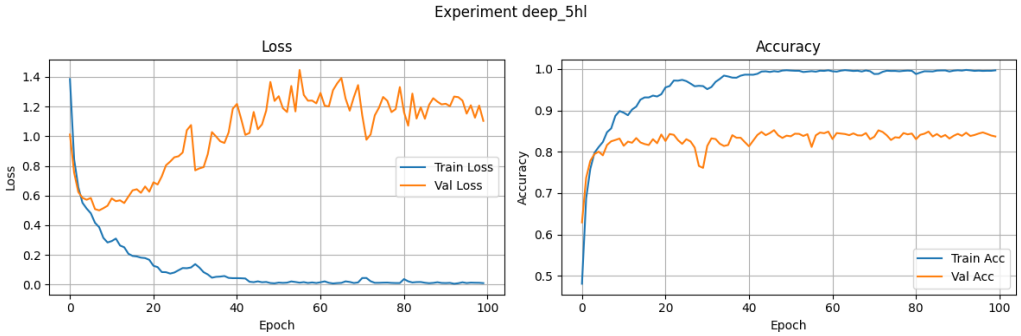

Figure 12: Deep 5HL (512,256,128,64,32) Accuracy/Loss plot

Metric	Value
Activation	ReLU
Batch Size	256
Loss	CrossEntropyLoss()
Optimizer	Adam
Optimizer Params	LR=0.001
Param Count	1379626
Regularization	None
Scheduler	None
Train Acc	99.68%
Train Loss	0.01
Train Time (sec.)	29.2
Training Samples	11964
Val Acc	83.70%
Val Loss	1.1

Table 11: Deep 5HL (512,256,128,64,32) Experiment Parameters

Observations: Overtraining is very visible in the above plots. Early stopping would prevent our validation accuracy from getting as worse as it did after the initial "convergence". (Look at the validation loss between epoch 20 and 100 in 12)

3.4 Regularization Experiments

Regularization aims at controlling overfitting in neural networks, especially when model capacity exceeds the complexity of the dataset. Techniques like dropout and weight decay constrain the network either by randomly suppressing activations or by penalizing large weights, forcing the model to learn more generalizable traits than just to "memorize the data". In practice, it narrows the gap between training and validation performance.

3.4.1 Dropout

Our part 2 MLP had 2 hidden layers. We can add 2 dropout layers after the activation functions of these two hidden layers to encourage feature learning instead of overfitting.

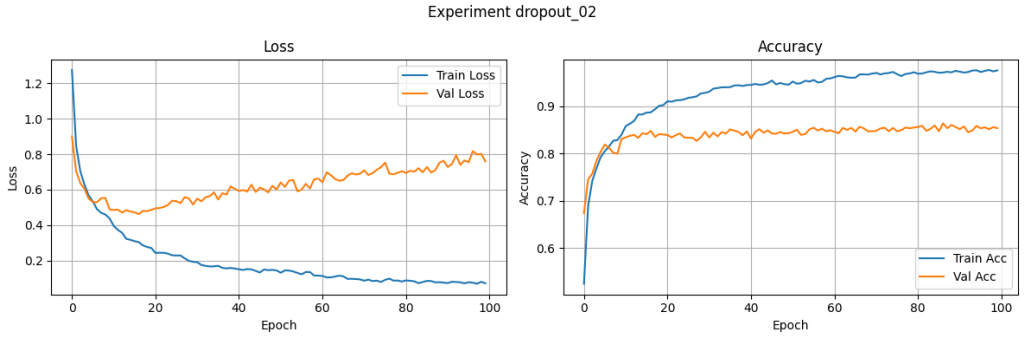

Figure 13: 2HL MLP + 0.2 dropouts Accuracy/Loss plot

Metric	Value
Activation	ReLU
Batch Size	256
Loss	CrossEntropyLoss()
Optimizer	Adam
Optimizer Params	LR=0.001
Param Count	310090
Regularization	Dropout 0.2
Scheduler	None
Train Acc	97.54%
Train Loss	0.07
Train Time (sec.)	21.4
Training Samples	11964
Val Acc	85.34%
Val Loss	0.76

Table 12: 2HL MLP + 0.2 dropouts Experiment Parameters

Observations: Adding dropout with a 0.2 rate to the 2HL MLP reduces overfitting effectively. Training accuracy drops to 97.5%, but validation accuracy improves to 85% with a loss of 0.76, suggesting better generalization. Overtraining is still clearly present (Validation loss of 13).

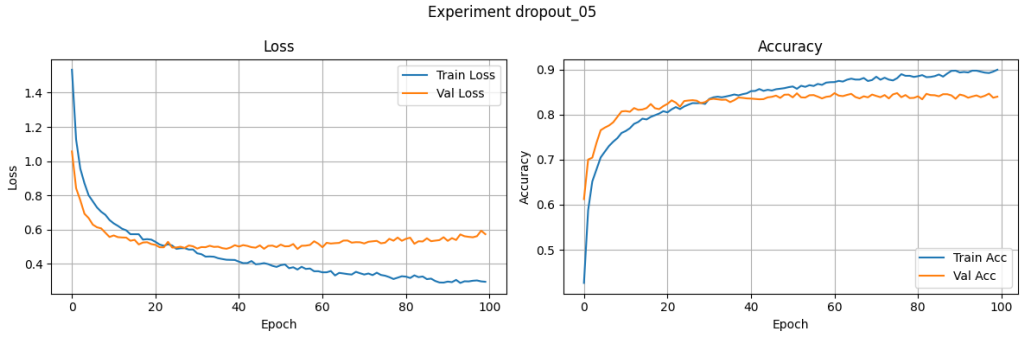

Figure 14: +2HL MLP + 0.5 dropouts Accuracy/Loss plot

Metric	Value
Activation	ReLU
Batch Size	256
Loss	CrossEntropyLoss()
Optimizer	Adam
Optimizer Params	LR=0.001
Param Count	310090
Regularization	Dropout 0.5
Scheduler	None
Train Acc	89.94%
Train Loss	0.29
Train Time (sec.)	30.5
Training Samples	11964
Val Acc	83.97%
Val Loss	0.57

Table 13: +2HL MLP + 0.5 dropouts Experiment Parameters

Observations: Increasing dropout to 0.5 produces a notable decline in training accuracy to 89% while not improving accuracy much more than 0.2 dropout did. We see less overtraining signs in the loss/acc plot which is desirable.

3.4.2 Weight Decay

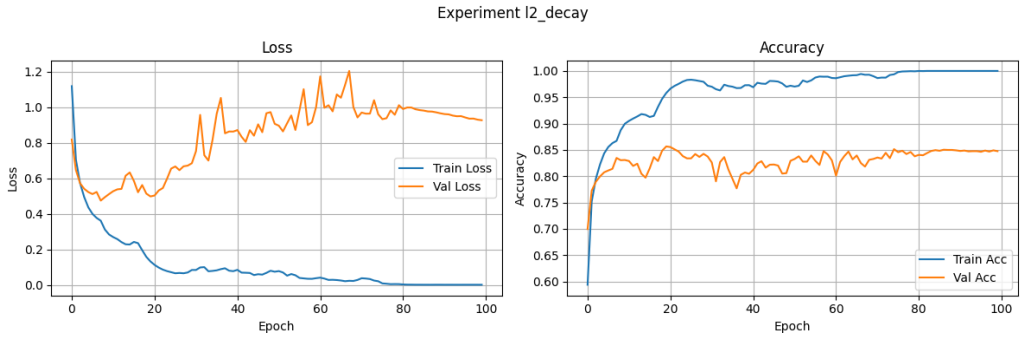

Figure 15: 2HL MLP + 1e-4 L2 Weight Decay Accuracy/Loss plot

Metric	Value
Activation	ReLU
Batch Size	256
Loss	CrossEntropyLoss()
Optimizer	Adam
Optimizer Params	LR=0.001
Param Count	310090
Regularization	L2 WD 1e-4
Scheduler	None
Train Acc	99.98%
Train Loss	0.0
Train Time (sec.)	30.32
Training Samples	11964
Val Acc	84.75%
Val Loss	0.93

Table 14: 2HL MLP + 1e-4 L2 Weight Decay Experiment Parameters

Observations: Generalization improves only modestly compared to vanilla 2HL MLP. Weight decay encourages smaller weights, limiting extreme parameter values, yet in this shallow MLP, the effect is less pronounced than dropout.

3.5 Scheduling Experiments

3.5.1 StepLR

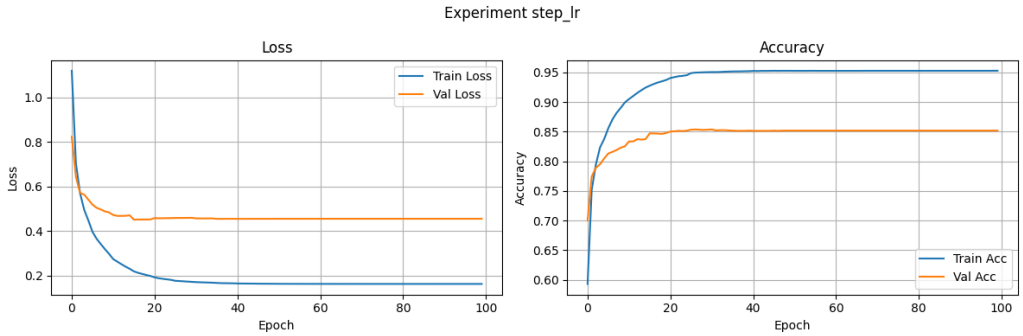

Figure 16: 2HL MLP + StepLR Accuracy/Loss plot

Metric	Value
Activation	ReLU
Batch Size	256
Loss	CrossEntropyLoss()
Optimizer	Adam
Optimizer Params	LR=0.001
Param Count	310090
Regularization	None
Scheduler	StepLRstep=5gamma=0.5
Train Acc	95.29%
Train Loss	0.16
Train Time (sec.)	23.33
Training Samples	11964
Val Acc	85.18%
Val Loss	0.46

Table 15: 2HL MLP + StepLR Experiment Parameters

Observations: Applying StepLR with a step size of 5 epochs and gamma=0.5 produces a modest improvement in generalization.

3.5.2 CosineAnnealingLR

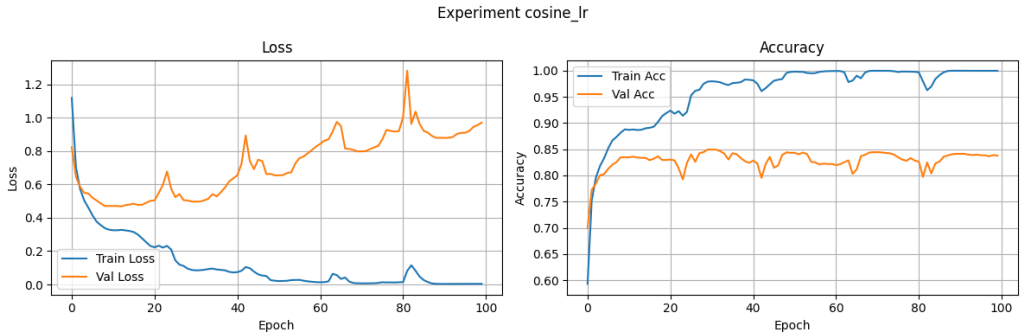

Figure 17: 2HL MLP + CosineAnnealingLR Accuracy/Loss plot

Metric	Value
Activation	ReLU
Batch Size	256
Loss	CrossEntropyLoss()
Optimizer	Adam
Optimizer Params	LR=0.001
Param Count	310090
Regularization	None
Scheduler	CosineAnnealingLRT_max=10
Train Acc	99.98%
Train Loss	0.0
Train Time (sec.)	24.32
Training Samples	11964
Val Acc	83.78%
Val Loss	0.97

Table 16: 2HL MLP + CosineAnnealingLR Experiment Parameters

Observations: These two experiments highlights that smooth schedulers improve convergence stability but do not inherently prevent overfitting.

3.6 Alternative Loss Functions

Switching the loss function affects both convergence dynamics and generalization behavior. CrossEntropyLoss is standard for classification, but alternatives like MSE can be used when targets are one-hot encoded.

3.6.1 MSE

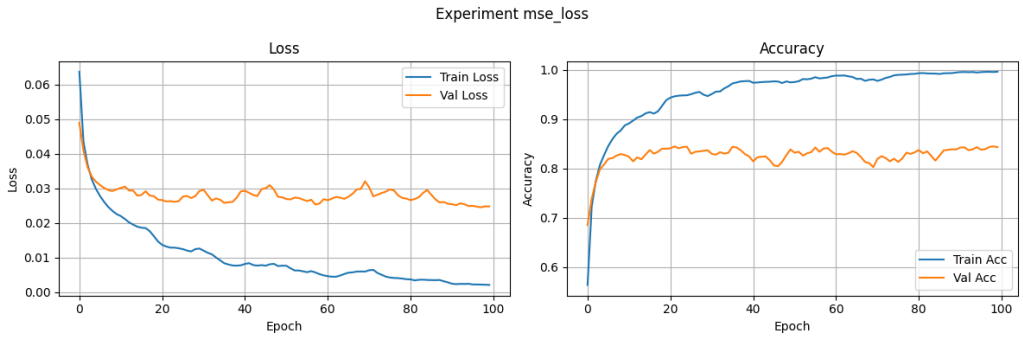

Figure 18: 2HL MLP + MSELoss (one-hot targets) Accuracy/Loss plot

Metric	Value
Activation	ReLU
Batch Size	256
Loss	mse_wrapper
Optimizer	Adam
Optimizer Params	LR=0.001
Param Count	310090
Regularization	None
Scheduler	None
Train Acc	99.63%
Train Loss	0.0
Train Time (sec.)	24.61
Training Samples	11964
Val Acc	84.32%
Val Loss	0.02

Table 17: 2HL MLP + MSELoss (one-hot targets) Experiment Parameters

Observations: Using MSELoss with one-hot encoded targets, the network achieves 99% training accuracy with a loss of near zero. Validation accuracy is at 84% with a loss of 0.02.

3.6.2 NLL

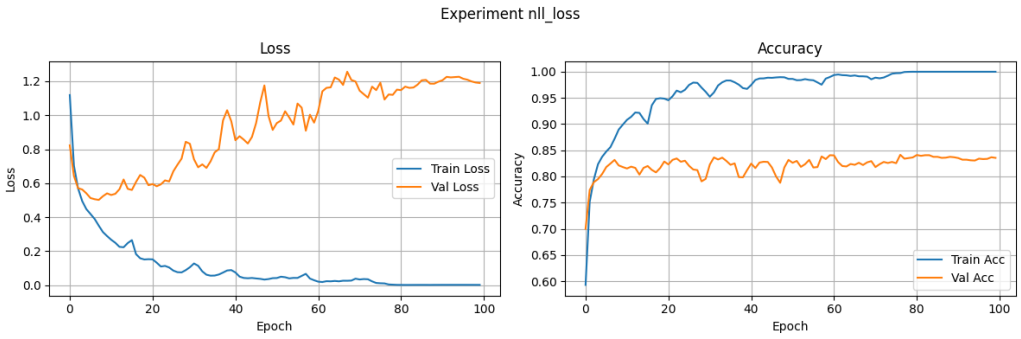

Figure 19: 2HL MLP + NLLLoss (wrapped) Accuracy/Loss plot

Metric	Value
Activation	ReLU
Batch Size	256
Loss	nll_wrapper
Optimizer	Adam
Optimizer Params	LR=0.001
Param Count	310090
Regularization	None
Scheduler	None
Train Acc	99.98%
Train Loss	0.0
Train Time (sec.)	23.24
Training Samples	11964
Val Acc	83.54%
Val Loss	1.19

Table 18: 2HL MLP + NLLLoss (wrapped) Experiment Parameters

Observations: Applying NLLLoss (wrapped for this setup) produces near-perfect training accuracy, but validation accuracy and loss is still very bad which shows overfitting. The jagged validation curves further confirm unstable generalization behavior.

4 Comparisons

Finally, we compare the evaluated metrics of all of the experiments we have conducted until now. We have sorted them descendingly by validation accuracy. By that measure, we can see that the experiment our combined method wins (with an ever so slightly lead over the batch-norm-only model). All of the experiments were done using batch size of 256 and initial LR=0.003,

trained for 100 epochs.

Exp. ID	Activation	Optimizer	Optimizer Params	Param Count	Regularization	Scheduler	Train Acc	Train Loss	Train Time (sec.)	Val Acc	Val Loss
Combined1	ReLU	AdamW	LR=0.001	310474	Dropout 0.5 BatchNorm 1e-3 Weight Decay	None	94.87%	0.16	29.46	86.47%	0.51
batchnorm	ReLU	Adam	LR=0.001	310474	BatchNorm	None	100.00%	0.0	36.18	86.35%	0.73
p3e22	ReLU	SGD	LR=0.001 momentum=0.9	310090	None	None	94.08%	0.2	27.07	85.69%	0.43
dropout_02	ReLU	Adam	LR=0.001	310090	Dropout 0.2	None	97.54%	0.07	21.4	85.34%	0.76
step_lr	ReLU	Adam	LR=0.001	310090	None	StepLR step=5 gamma=0.5	95.29%	0.16	23.33	85.18%	0.46
medium_3hl	ReLU	Adam	LR=0.001	644170	None	None	99.98%	0.0	23.71	85.10%	1.17
p3e12	LeakyReLU	Adam	LR=0.001	310090	None	None	99.98%	0.0	20.57	84.87%	1.15
l2_decay	ReLU	Adam	LR=0.001	310090	L2 WD 1e-4	None	99.98%	0.0	30.32	84.75%	0.93
mse_loss	ReLU	Adam	LR=0.001	310090	None	None	99.63%	0.0	24.61	84.32%	0.02
p3e24	ReLU	RMSprop	LR=0.001	310090	None	None	99.98%	0.0	23.67	84.17%	1.2
2HL_standard	ReLU	Adam	LR=0.001	310090	None	None	99.98%	0.0	19.84	84.05%	1.23
dropout_05	ReLU	Adam	LR=0.001	310090	Dropout 0.5	None	89.94%	0.29	30.5	83.97%	0.57
cosine_lr	ReLU	Adam	LR=0.001	310090	None	CosineAnnealingLR T_max=10	99.98%	0.0	24.32	83.78%	0.97
deep_5hl	ReLU	Adam	LR=0.001	1379626	None	None	99.68%	0.01	29.2	83.70%	1.1
p3e14	Sigmoid	Adam	LR=0.001	310090	None	None	99.97%	0.0	23.96	83.62%	0.72
nll_loss	ReLU	Adam	LR=0.001	310090	None	None	99.98%	0.0	23.24	83.54%	1.19
2HL_small_bs	ReLU	Adam	LR=0.001	310090	None	None	97.94%	0.09	55.94	82.88%	1.61
shallow_1hl	ReLU	Adam	LR=0.001	151242	None	None	99.84%	0.01	19.34	82.57%	1.07
p3e13	Tanh	Adam	LR=0.001	310090	None	None	98.37%	0.05	22.06	82.37%	0.92
p3e21	ReLU	SGD	LR=0.001	310090	None	None	71.86%	0.8	21.25	71.68%	0.81

Table 19: The ultimate comparison